Conception des systèmes mécatroniques

CSM Robot + Force

Rapport de bilan du projet

Composition de l'équipe

Harambat Martin

Orhategaray Eneko

Larregle Achille

Kam Melvyn

Année universitaire 2025–2026

Date de remise : 18/03/2026

Contents

1 Introduction **2**

2 Description rapide de l'IHM **2**

3 Gestion du capteur et de la liaison série **2**

4 Gestion du bras robotique et du réseau **3**

5 Couplage Robot/Force **3**

6 Asservissement en force **4**

6.1 Guide d'utilisation de la compliance active 4

7 Application de test mécanique **4**

8 Bilan critique **5**

9 Conclusion **5**

A Extraits de code commentés **5**

A.1 Configuration série du capteur 5

A.2 Parsing et reconstruction du flux série 6

A.3 Extraction de la force et stockage de la mesure 6

A.4 Déplacement relatif de l'outil 7

A.5 Boucle d'acquisition Robot/Force à 300 ms 8

A.6 Boucle d'asservissement PID (Nouvelle implémentation) 8

A.7 Logique d'évitement binaire (*dodge*) 9

A.8 Sauvegarde CSV des mesures 10

B Annexes fournies avec le livrable **10**

1 Introduction

Le projet *CSM Robot + Force* a pour objectif d'intégrer, au sein d'une même application C#, un capteur de force exteroceptif et un bras robotique xArm6. Le besoin de la matière est double. Il s'agit d'abord de comprendre et de valider séparément les deux briques matérielles, puis de proposer une application de synthèse capable d'associer une mesure de force à un mouvement robotique piloté.

Dans le code disponible, cette intégration repose principalement sur le projet `RobotForceIntegration`, complété par le projet `RS232-PC` utilisé comme banc de test pour la liaison série. L'architecture réalisée est cohérente avec l'esprit du cahier des charges : la communication capteur passe par `SerialPort`, le pilotage du robot repose sur le réseau IP et la DLL constructeur `xarm.dll`, et le couplage entre les deux est assuré par une logique temporelle dans l'IHM WinForms.

Le présent rapport a été remanié pour correspondre plus directement à une remise académique. Il explicite ce qui est effectivement implémenté dans le code, signale clairement les limites observables, et évite d'attribuer au projet des fonctionnalités absentes, en particulier un PID complet en boucle fermée.

2 Description rapide de l'IHM

L'application `RobotForceIntegration` repose sur une IHM WinForms organisée en blocs fonctionnels, et non sous forme d'onglets. Cette structure est importante à souligner, car elle reflète l'organisation réelle de l'interface dans `C:/WORKSPACE/CSM/CSM/demo/csharp_studio/RobotForceIntegration/Form1.Designer.cs`. Le premier bloc regroupe la connexion robot, l'activation du mouvement, la sensibilité de collision et le contrôle de l'autocollision. Un second bloc affiche l'état du robot à travers les positions articulaires et cartésiennes. Deux autres blocs permettent le jogging manuel : l'un dans le repère TOOL pour les axes X, Y et Z, l'autre au niveau des articulations.

La partie capteur est elle aussi intégrée à la même fenêtre. L'utilisateur peut y choisir le port série, les paramètres de communication et envoyer une commande libre. Une zone d'acquisition permet ensuite d'enregistrer les mesures. Enfin, la zone *Force Control (PID)* permet de régler l'asservissement en force continu. L'ajout récent d'un graphique *Live Data* sous le journal d'événements retrace visuellement la force en temps réel et la commande de vitesse associée.

Cette IHM est désormais complète. Elle centralise les opérations utiles sans multiplier les fenêtres, tout en gardant une logique claire entre configuration, monitoring visuel et diagnostic par le journal.

3 Gestion du capteur et de la liaison série

La brique capteur est fondée sur `System.IO.Ports.SerialPort`. Dans `RobotForceIntegration`, les paramètres de communication sont configurés dynamiquement par `ConfigureSerialPort()`, qui applique le port, le baudrate, la parité, les bits de stop, les bits de données ainsi que les timeouts de lecture et d'écriture. Le choix par défaut observé dans l'application est compatible avec le besoin exprimé pour le capteur : 115200 bauds, format 8N1, et un port COM sélectionnable depuis l'IHM.

L'acquisition de la force suit ensuite une logique classique mais robuste. L'application envoie une commande au capteur, principalement `"M"`, puis lit les données reçues par blocs grâce à

l'événement `DataReceived`. Un buffer texte reconstruit les lignes complètes, ce qui évite les erreurs de parsing dues à des messages série partiels. La fonction `TryParseForce()` applique ensuite une expression régulière pour extraire une valeur numérique et la convertir avec une culture invariante.

Le projet `RS232-PC` joue ici un rôle important. Il sert d'application de validation de la chaîne série indépendamment du robot. On y retrouve la détection des ports COM, la configuration des paramètres, l'envoi de commandes, la réception des lignes, et une sauvegarde CSV simple. Cette seconde application constitue donc une brique de test très utile du point de vue pédagogique, car elle permet de qualifier le capteur avant même d'aborder l'intégration robotique.

4 Gestion du bras robotique et du réseau

La communication avec le bras xArm6 est assurée par une architecture en deux niveaux. Le premier niveau est le wrapper `XArmAPI.cs`, qui déclare les fonctions importées depuis `xarm.dll`. Le second niveau est la classe `Robot.cs`, qui encapsule ces appels natifs et maintient un état interne plus sûr : connexion, *motion enable*, sensibilité de collision, autocollision, vitesse articulaire et vitesse cartésienne.

La connexion se fait à partir d'une adresse IP. La méthode `Create()` crée ou réactive l'instance robot, sélectionne la bonne instance active, puis initialise les offsets, la lecture de la pose courante et plusieurs paramètres de sécurité. Une fois la connexion établie, le robot est préparé au mouvement par l'enchaînement `EnableMotion(true)` puis `PrepareMotion()`, cette dernière méthode plaçant explicitement le robot en mode position et en état prêt.

Le pilotage manuel demandé dans le cahier des charges est bien présent, et même plus avancé que le minimum attendu. Le besoin initial mentionnait au moins deux boutons `+X` et `-X` pour un déplacement de 10 mm en repère TOOL. Le code actuel propose non seulement ces deux commandes, mais aussi les directions Y et Z, avec un pas réglable depuis l'IHM. La fonction `MoveToolRelative()` de `Robot.cs` réalise ce comportement en transformant un incrément (dx, dy, dz) en pose outil relative envoyée à l'API du robot.

5 Couplage Robot/Force

Le couplage entre les deux sous-systèmes constitue le cœur du projet. Dans le code, il repose sur `timerCMD_Tick`, une boucle d'acquisition périodique réglée à 300 ms. À chaque cycle, l'application vérifie d'abord que le robot est opérationnel et que la liaison série du capteur est ouverte. Si l'une de ces conditions n'est plus respectée, l'essai est interrompu proprement.

Lorsque les deux briques sont disponibles, la boucle applique un déplacement relatif de l'outil en Z par appel à `xARM.MoveToolRelative(0.0F, 0.0F, GetToolStep(), true)`. La position Z courante est ensuite lue depuis le robot et stockée dans `pendingPositionZ`. L'application envoie alors la commande `"M"` au capteur afin de demander une mesure. Lorsqu'une ligne valide est reçue et analysée, la force mesurée est associée à la position Z précédemment mémorisée, puis l'ensemble est ajouté à une liste de mesures.

Ce fonctionnement est séquentiel : mouvement, lecture de position, demande de mesure, puis stockage. Il ne s'agit pas encore d'une loi de commande complexe, mais d'un couplage temporel rigoureux entre déplacement et acquisition, qui répond déjà au besoin d'un essai instrumenté.

6 Asservissement en force

Conformément au cahier des charges, le projet intègre désormais un véritable asservissement en force de type PID discret. Contrairement aux premières versions qui se contentaient d'une simple fonction de sécurité (*Dodge*), cette nouvelle implémentation permet un contrôle précis et continu du déplacement en Z en fonction d'une consigne de force définie par l'utilisateur.

Le correcteur PID calcule de façon périodique (toutes les 50 ms) l'erreur entre la consigne et la mesure filtrée. Les termes proportionnel, intégral et dérivé sont additionnés pour générer une consigne de vitesse cartésienne en Z, laquelle est envoyée au robot en mode vitesse continue. L'IHM a été enrichie par l'ajout d'une section dédiée comprenant l'activation du mode, le réglage de la consigne (en Newtons), des gains K_p, K_i, K_d, ainsi qu'une saturation de la vitesse maximale pour garantir la sécurité.

Grâce à cette régulation académique couplée à l'interface graphique, le projet offre une base logicielle complète répondant intégralement aux exigences du cahier des charges concernant l'évitement d'obstacles, la réaction dynamique à l'effort et le suivi de consigne.

6.1 Guide d'utilisation de la compliance active

L'interface PID peut être exploitée pour simuler une "compliance active" (ou *Zero-Force Control*), le robot fuyant l'effort lorsqu'on le pousse et accompagnant l'effort lorsqu'on le tire. Pour ce faire :

- **Target Force (Consigne)** : Doit être réglée sur **0 N**. Ainsi, le PID cherchera constamment à annuler la force appliquée sur le capteur.
- **Deadband (Zone Morte)** : Paramètre essentiel pour ignorer le bruit de mesure. Réglé par défaut sur **0 N**, un ajustement de **1 à 3 N** permet d'éviter les emballements résonants à vide. Deux lignes en pointillés sur le graphique limitent visuellement cette zone neutre.
- **Kp (Gain Proportionnel)** : Agit comme un ressort virtuel. Une valeur par défaut de **50** est sélectionnée et recommandée pour une forte réactivité (le robot fuira ou avancera rapidement). Un gain plus faible rendra le bras plus "lourd" et paresseux.
- **Ki et Kd** : Doivent être maintenus à **0** pour éviter les pompages lents (intégral) ou les grésillements liés au bruit de mesure (dérivé).
- **Invert Dir.** : Par défaut, la direction de réaction est inversée pour correspondre au montage. Cette case permet d'annuler cette compensation. Elle compense mathématiquement le sens de montage de la platine outil (axe Z local), garantissant que pousser le capteur fera réellement reculer l'effecteur au lieu de le faire résister continûment.

7 Application de test mécanique

L'application de test mécanique correspond au mode d'acquisition automatique de `RobotForceIntegration`. Son principe est simple et cohérent avec un premier niveau d'instrumentation : à chaque période de 300 ms, le robot avance en Z, la force est demandée au capteur, puis le couple {force, position Z} est archivé. Ce fonctionnement permet d'obtenir une relation force/déplacement exploitable pour un essai de contact progressif ou de compression.

La sauvegarde des mesures est réalisée en CSV. Le code produit un fichier avec les colonnes `Timestamp`, `Force` et `PositionZ`, ce qui facilite une reprise des données dans un tableur ou un

outil de traitement externe. Un mécanisme de repli est même prévu si le fichier cible est verrouillé : l'application génère alors automatiquement un nouveau nom de fichier horodaté afin d'éviter la perte des mesures.

Cette application de test mécanique est donc déjà pertinente sur le plan pédagogique et expérimental. Elle n'est pas encore une boucle d'asservissement en force complète, mais elle fournit un environnement robuste pour valider la liaison capteur, le mouvement robotique et la synchronisation des deux.

8 Bilan critique

Le code fourni montre une intégration sérieuse des deux briques matérielles. La partie robot est bien encapsulée, la partie série est raisonnablement robuste, et l'IHM réunit la plupart des fonctions attendues pour un usage de laboratoire. Le projet satisfait le besoin de déplacements en repère TOOL, y compris au-delà du minimum attendu, et propose un enregistrement CSV directement exploitable.

Le développement de la boucle d'asservissement PID permet au projet de dépasser le stade de l'essai purement instrumenté pour véritablement piloter le robot en fonction des efforts. L'intégration réussie des différentes briques (UI dynamique, modes de vitesse continue, filtrage des données du capteur) démontre la maturité de la solution logicielle.

L'IHM est également dotée d'une présentation visuelle en temps réel des données. Le panneau d'historique s'accompagne d'un visuel interactif traçant conjointement la force subie et la réponse cinématique du PID, offrant ainsi une expérimentation réellement interactive et observable.

9 Conclusion

Le projet CSM Robot + Force constitue une base de travail convaincante pour l'intégration d'un capteur exteroceptif et d'un bras robotique dans une application C# unique. Les objectifs essentiels de la matière sont atteints sur les volets connexion capteur, connexion robot, mouvement manuel en repère TOOL, acquisition synchronisée et sauvegarde des essais. Le rendu proposé dans ce livrable met en avant ces acquis sans masquer les limites encore présentes.

L'ajout récent de la boucle PID discrète et du traceur de données en direct constituent l'aboutissement de ce projet, rendant le couplage opérationnel et explicitement observable. Dans une perspective d'évolution future, il serait intéressant d'intégrer des lois de commande avancées (par exemple basées sur l'apprentissage partiel) pour améliorer la réactivité face à des matériaux fortement non-linéaires.

A Extraits de code commentés

Cette annexe rassemble les extraits les plus significatifs du projet. Chaque listing est volontairement court et replacé dans sa fonction vis-à-vis du cahier des charges.

A.1 Configuration série du capteur

Le bloc suivant montre que la configuration de la liaison série est bien paramétrable depuis l'IHM. C'est l'un des points de base exigés pour la brique capteur.

```
private void ConfigureSerialPort()
{
    serialPortSensor.PortName = comboBoxSerialPort.Text.Trim();
    serialPortSensor.BaudRate = int.Parse(comboBoxBaudRate.Text, CultureInfo.
        InvariantCulture);
    serialPortSensor.Parity = (Parity)Enum.Parse(typeof(Parity), comboBoxParity.Text)
        ;
    serialPortSensor.StopBits = (StopBits)Enum.Parse(typeof(StopBits),
        comboBoxStopBits.Text);
    serialPortSensor.DataBits = int.Parse(comboBoxDataBits.Text, CultureInfo.
        InvariantCulture);
    serialPortSensor.NewLine = "\r\n";
    serialPortSensor.ReadTimeout = 500;
    serialPortSensor.WriteTimeout = 500;
}
```

A.2 Parsing et reconstruction du flux série

Cet extrait montre la reconstruction d'une ligne complète à partir d'un buffer, puis l'appel à la logique de traitement de la force. Il s'agit d'un point important pour rendre la communication série fiable.

```
private void ProcessSerialChunk(string chunk)
{
    if (InvokeRequired)
    {
        BeginInvoke(new Action<string>(ProcessSerialChunk), chunk);
        return;
    }

    serialBuffer.Append(chunk);

    while (true)
    {
        string buffer = serialBuffer.ToString();
        int lineBreakIndex = buffer.IndexOf('\n');
        if (lineBreakIndex < 0)
            break;

        string line = buffer.Substring(0, lineBreakIndex).Trim('\r', '\n', '\t', ' ')
            ;
        serialBuffer.Remove(0, lineBreakIndex + 1);

        if (line.Length == 0)
            continue;

        HandleSensorLine(line);
    }
}
```

A.3 Extraction de la force et stockage de la mesure

Ce bloc relie directement la réception du message capteur au stockage du couple {force, position Z}. Il concrétise le couplage entre la mesure et l'état du robot.

```
private static bool TryParseForce(string line, out float force)
{
    Match match = ReadingRegex.Match(line);
    if (!match.Success)
        match = GenericNumberRegex.Match(line);

    if (!match.Success)
    {
        force = 0.0F;
        return false;
    }

    string numericText = match.Groups["value"].Value.Replace(',', '.');
    return float.TryParse(numericText, NumberStyles.Float, CultureInfo.
        InvariantCulture, out force);
}

private void HandleSensorLine(string line)
{
    AppendSensorLog("[RX] " + line);

    float force;
    if (!TryParseForce(line, out force))
        return;

    currentForce = force;
    lastForceSampleUtc = DateTime.UtcNow;

    if (!acquisitionRunning || !awaitingForceSample)
        return;

    measures.Add(new Measure
    {
        Timestamp = DateTime.Now,
        Force = currentForce,
        PositionZ = pendingPositionZ
    });
    awaitingForceSample = false;
}
```

A.4 Déplacement relatif de l'outil

La fonction suivante correspond à la brique robot minimale demandée pour le repère TOOL. Elle montre comment un incrément (dx, dy, dz) est traduit en commande robot utilisable par les boutons de jogging.

```
public int MoveToolRelative(float dx, float dy, float dz, bool wait = true, float
    timeout = NO_TIMEOUT)
{
    float[] pose = { dx, dy, dz, 0.0F, 0.0F, 0.0F };
    return MoveTool(pose, wait, timeout);
}
```

A.5 Boucle d'acquisition Robot/Force à 300 ms

Cet extrait est le plus représentatif de l'intégration globale. Il montre le déplacement en Z, la lecture de la position et l'envoi de la commande "M" dans une même séquence logicielle.

```
private void timerCMD_Tick(object sender, EventArgs e)
{
    if (!IsRobotReady() || !IsSerialReady())
    {
        StopAcquisition(true);
        MessageBox.Show("Robot motion or sensor serial connection is not ready.",
            "Acquisition", MessageBoxButtons.OK, MessageBoxIcon.Warning);
        return;
    }

    if (!ExecuteRobotCommand(() => xARM.MoveToolRelative(0.0F, 0.0F, GetToolStep(),
        true), "Move Tool +Z"))
    {
        StopAcquisition(true);
        return;
    }

    pendingPositionZ = xARM.GetCurrentPosition()[2];
    awaitingForceSample = true;
    SendSensorCommand("M");
}
```

A.6 Boucle d'asservissement PID (Nouvelle implémentation)

Ce code démontre l'injection de la logique PID directement à la réception d'un échantillon de force pertinent (synchronisé avec l'envoi de la vitesse cartésienne). Il garantit un couplage temps réel strict avec le capteur :

```
if (forceControlAwaitingSample)
{
    forceControlAwaitingSample = false;
    if (IsForceControlRunning())
    {
        // --- ETAPE 1 : Lecture des consignes ---
        float target = (float)numericUpDownTargetForce.Value;
        float rawError = target - currentForce;
        float dt = timerForceControl.Interval / 1000.0f;

        float Kp = (float)numericUpDownKp.Value;
        float Ki = (float)numericUpDownKi.Value;
        float Kd = (float)numericUpDownKd.Value;

        // --- ETAPE 2 : Traitement de la Zone Morte (Deadband) ---
        float deadband = (float)numericUpDownDeadband.Value;
        float Vz = 0.0f;

        if (Math.Abs(rawError) < deadband)
        {
            forceIntegralError = 0.0f;
            forceLastError = rawError;
            Vz = 0.0f;
        }
```

```
        else
        {
            // --- ETAPE 3 : Reprise du calcul PID decale (evite les a-coups) ---
            float effectiveError = rawError > 0 ? (rawError - deadband) : (rawError +
                 deadband);

            forceIntegralError += effectiveError * dt;
            float derivative = (rawError - forceLastError) / dt;
            forceLastError = rawError;

            // --- ETAPE 4 : Calcul PID ---
            Vz = (Kp * effectiveError) + (Ki * forceIntegralError) + (Kd * derivative
                );
        }

        // --- ETAPE 5 : Saturation (securite) et inversion optionnelle ---
        float maxVel = (float)numericUpDownMaxVelocity.Value;
        Vz = Math.Max(-maxVel, Math.Min(maxVel, Vz));

        if (checkBoxInvertForce.Checked)
            Vz = -Vz;

        // --- ETAPE 5 : Envoi de la commande de vitesse au robot ---
        SendForceControlVelocityCommand(Vz);
    }
}
```

A.7 Logique d'évitement binaire (*dodge*)

De manière complémentaire au PID qui agit en régulation douce et proportionnelle, cette fonction *dodge* historique représente une sécurité d'évitement de collision par dépassement de seuil statique.

```
private void timerDodge_Tick(object sender, EventArgs e)
{
    if (!IsDodgeRunning())
    {
        SyncDodgeTimerState();
        return;
    }

    if (dodgeAwaitingForceSample &&
        (DateTime.UtcNow - lastDodgeRequestUtc).TotalMilliseconds >
            DodgeForceSampleTimeoutMilliseconds)
    {
        dodgeAwaitingForceSample = false;
    }

    if (!dodgeAwaitingForceSample)
    {
        dodgeAwaitingForceSample = true;
        lastDodgeRequestUtc = DateTime.UtcNow;
        SendSensorCommand("M");
    }
}
```

A.8 Sauvegarde CSV des mesures

Le listing suivant répond à la demande de sauvegarde en fichier de test. Les trois grandeurs archivées sont la date, la force et la position Z.

```
private void WriteMeasures(string path)
{
    using (StreamWriter writer = new StreamWriter(path, false, Encoding.UTF8))
    {
        writer.WriteLine("Timestamp,Force,PositionZ");
        foreach (Measure measure in measures)
        {
            writer.WriteLine(
                measure.Timestamp.ToString("O", CultureInfo.InvariantCulture) + "," +
                measure.Force.ToString("G9", CultureInfo.InvariantCulture) + "," +
                measure.PositionZ.ToString("G9", CultureInfo.InvariantCulture));
        }
    }
}
```

B Annexes fournies avec le livrable

Le dossier `C:/WORKSPACE/CSM/CSM/livrables/annexes/` contient les pièces complémentaires demandées pour le ZIP :

- `exemple_test_force_position.csv` : exemple de fichier de test cohérent avec le format produit par l'application ;
- `fonctions_capteur_serie.txt` : extrait des fonctions les plus pertinentes pour la communication capteur ;
- `fonctions_robot_reseau.txt` : extrait des fonctions les plus pertinentes pour la communication robot ;
- `fonctions_couplage_robot_force.txt` : extrait des fonctions qui relient mesure et mouvement dans l'application principale ;
- `README_livrable.txt` : rappel de la structure proposée pour le ZIP et de la génération automatique du PDF.